

Optimización por Enjambre de Partículas (PSO) para el Problema del Viajante de Comercio y Comparativa ACO vs PSO

Informe Técnico Oficial: Representación por Claves Aleatorias, Escalamiento Multiescala y Evaluación Head-to-Head

Nicolas Jacobo Mondragón Rey
Nicolas Emilio Morantes Puentes
Santiago Jose Rodríguez Gutierrez

Curso de Introducción a la Inteligencia Artificial 2026
Inteligencia de Enjambre y Metaheurísticas (Sesión 09)

Septiembre 2026

Resumen

Se presenta la formulación, diseño computacional y análisis empírico del algoritmo de Optimización por Enjambre de Partículas (*Particle Swarm Optimization*, PSO) aplicado al Problema del Viajante de Comercio (*Traveling Salesperson Problem*, TSP) sobre instancias euclidianas uniformes de $N \in \{20, 2.000, 200.000\}$ ciudades. Dado que el PSO canónico opera sobre espacios **continuos** y el TSP es combinatorio, se adopta la codificación por *random keys* ($\text{tour} = \text{argsort}(x)$, $x \in [0, 1]^N$) con las ecuaciones estándar de velocidad de tres componentes (inercia, memoria personal y memoria social), inercia decreciente $w : 0.90 \rightarrow 0.40$, cota de velocidad $v_{\text{máx}} = 0.20$ y parada por S iteraciones sin mejora. La escala masiva se aborda con descomposición jerárquica en grilla 15×15 paralelizada con OpenMP (≈ 225 celdas de ≈ 900 ciudades). Con $S = N$ partículas en el régimen serie, la instancia de 200.000 ciudades converge a un circuito hamiltoniano válido en 4.4 s con un pico de 11.6 MB ($\approx 13.800\times$ menor a la matriz densa de 160 GB). La calidad, sin embargo, revela el límite de la representación: el enjambre colapsa prematuramente sobre claves casi idénticas y el gap frente al estimador de Beardwood crece de +87% ($N = 20$) a +1970% ($N = 200.000$), demostrando cuantitativamente que un optimizador continuo mal adaptado no compite en un problema combinatorio. En la segunda parte se confronta este PSO con nuestra implementación ACO bajo protocolo estricto (mismas instancias, mismo presupuesto, multi-semilla): el ACO vence en calidad en las tres escalas (exceso 24%/0.2%/0.1% frente a 90%/850%/1294%) a cambio de 2–5× más tiempo, empatando en memoria (< 12 MB).

Parte I

Informe PSO

1. Definición del Problema y Complejidad Combinatoria

El Problema del Viajante de Comercio (TSP) consiste en determinar el ciclo hamiltoniano de coste euclidiano mínimo que visite un conjunto discreto de N vértices exactamente una vez y retorne al nodo de origen:

$$\min_{\pi \in \Pi} L(\pi) = \sum_{i=1}^{N-1} d(c_{\pi(i)}, c_{\pi(i+1)}) + d(c_{\pi(N)}, c_{\pi(1)}) \quad (1)$$

donde $c_i = (x_i, y_i) \in [0, 1]^2$ y d es la distancia euclidiana. En un grafo completo no dirigido, $|E| = N(N - 1)/2$ y el número de tours distintos es $|\Pi| = (N - 1)!/2$: para $N = 20$ ya son $\approx 6 \times 10^{16}$ (fuerza bruta imposible) y para $N = 2.000$ del orden de 10^{5735} . La matriz densa de $N = 200.000$ exigiría 160 GB en `float32`, de modo que solo son viables métodos con memoria subcuadrática.

2. Análisis Matemático del Consumo de Memoria

El enjambre almacena tres vectores reales por partícula (posición X , velocidad V y mejor personal P), es decir $3 \cdot S \cdot N$ flotantes:

$$M_{\text{enjambre}} = 12 \cdot S \cdot N \text{ bytes (float32)}. \quad (2)$$

Con $S = N = 2.000$ el enjambre ocupa 48 MB: es el único punto donde este PSO consume **más** memoria que el ACO disperso (feromona de 64 KB). En el régimen jerárquico cada celda (≈ 900 ciudades, $S = 32$) demanda ≈ 350 KB por arreglo, con picos medidos de 11.6 MB totales. No se materializa jamás ninguna matriz $N \times N$: las distancias se evalúan al vuelo y la decodificación reutiliza un único buffer de índices $O(N)$.

3. Formulación del Enjambre de Partículas (PSO)

Cada partícula i conserva posición x_i , velocidad v_i , mejor personal p_i y el enjambre comparte el mejor global g . La actualización estándar es:

$$v_i(t+1) = w v_i(t) + c_1 r_1 \odot [p_i - x_i(t)] + c_2 r_2 \odot [g - x_i(t)], \quad x_i(t+1) = x_i(t) + v_i(t+1), \quad (3)$$

con $r_1, r_2 \sim U[0, 1]$ por partícula. Los tres sumandos son inercia (continuidad), componente cognitiva (memoria individual) y componente social (memoria colectiva). Se aplica recorte $x \in [0, 1]$ y cota $|v| \leq v_{\text{máx}}$, y la inercia decrece linealmente $w : 0.90 \rightarrow 0.40$ para explorar al inicio y estabilizar al final. Parámetros: $c_1 = 1.4$, $c_2 = 1.6$ (ejemplo calculado de la guía), $v_{\text{máx}} = 0.20$, p_i y g por minimización, parada por máximo de iteraciones o S iteraciones sin mejora.

4. Adaptación al TSP: Claves Aleatorias

La posición continua se decodifica por ordenamiento: $\text{tour} = \text{argsort}(x)$, con costo $O(N \log N)$ por evaluación más $O(N)$ de la longitud. La velocidad inicia en cero y la primera posición se copia como mejor personal. Esta codificación es el talón de Aquiles del método: mover las *claves* hacia las claves de g **no** acerca el *tour* al tour de g , pues infinitos vectores codifican la misma permutación y vecinos en $[0, 1]^N$ decodifican a tours arbitrariamente lejanos. El enjambre, por tanto, colapsa sobre una región de claves y el orden deja de correlacionarse con la geografía: estancamiento local sin convergencia al óptimo.

5. Metodología Experimental y Regímenes de Escala

Régimen	N	S (partículas)	T (iters)	Parada S
Serie	20	30	300	50
Serie + 2-opt	2.000	N	100	15
Jerárquico (15×15)	200.000	32/celda	40	—

$S = N$ guarda simetría con el ACO ($m = n$). Las instancias son idénticas a las del ACO (mismo generador y semilla; verificado por `diff`), de modo que los tours son directamente comparables. Las celdas usan semilla propia (determinismo bajo OpenMP) y se cosen por vecino más cercano sobre centroides.

6. Resultados Experimentales y Discusión Técnica

6.1. Traza de Ejecución y Validación de la Salida (./bin/pso)

```
./bin/pso -n 20
Params: S=30 part. T=300 w=[0.90,0.40] c1=1.40 c2=1.60 vmax=0.20 S_stop=50
Densa teorica: 0.000 GB | enjambre: 0.01 MB
it 1/300 mejor-global=8.12 mejor-it=8.12 w=0.900
it 2/300 mejor-global=7.40 mejor-it=7.40 w=0.898
it 3/300 mejor-global=6.52 mejor-it=6.52 w=0.897
...
parada temprana en it 134 (S=50 sin mejora)
Evaluaciones totales = 9000 (S x T)
Referencia NN greedy: 4.33
Tour valido: SI | longitud = 5.95
Tiempos: PSO/jer=0.003s 2opt=0.000s TOTAL=0.003s
Memoria: RSS actual 3708 -> 3780 KB (+0.1 MB) | peak VmHWM 3708 -> 3780 KB

./bin/pso -n 2000 (extracto)
Params: S=2000 part. T=100 w=[0.90,0.40] c1=1.40 c2=1.60 vmax=0.20 S_stop=15
it 2/100 mejor-global=1009.93 mejor-it=1019.13 w=0.895
it 3/100 mejor-global=1009.93 mejor-it=1012.65 w=0.890
it 4/100 mejor-global=1009.93 mejor-it=1016.49 w=0.885
parada temprana en it 15 (S=15 sin mejora)
Tour valido: SI | longitud = 383.54 (el 2-opt rescata de ~1010 a 383)
Tiempos: PSO/jer=5.587s 2opt=0.001s TOTAL=5.599s
Memoria: RSS actual 3764 -> 3892 KB (+0.1 MB) | peak VmHWM 3764 -> 50644 KB

./bin/pso -n 200000 (extracto)
Params: S=32 part. T=40 w=[0.90,0.40] c1=1.40 c2=1.60 vmax=0.20 S_stop=0
Grilla 15x15 -> 225 celdas
Evaluaciones totales = 288000 (n_celdas x S x T)
Tour valido: SI | longitud = 6591.53
Tiempos: PSO/jer=4.533s 2opt=0.000s TOTAL=4.545s
Memoria: RSS actual 3728 -> 9716 KB (+6.0 MB) | peak VmHWM 3728 -> 11316 KB
```

6.1.1. Análisis de Métricas de la Salida

- **Instancia $N = 20$:** el récord desciende por escalones ($8.12 \rightarrow 7.40 \rightarrow 6.52 \rightarrow 5.95$) con w decayendo visiblemente ($0.900 \rightarrow 0.678$); cada escalón es un tour estrictamente mejor. La parada $S = 50$ corta en la iteración 134 de 300 (55% de cómputo ahorrado). El resultado (5.95) queda **peor** que el voraz NN (4.33): el enjambre no supera ni a la heurística más simple.
- **Instancia $N = 2.000$:** con $S = N = 2.000$ partículas, el mejor de la inicialización (≈ 1010 , un tour casi aleatorio) **jamás mejora** en 14 iteraciones y la parada $S = 15$ dispara en el primer instante posible. Todo el mérito ($1010 \rightarrow 383$) corresponde al pulido 2-opt final de 0.001 s: el PSO propone y el 2-opt dispone. El pico de 50.6 MB es el enjambre $2000 \times 2000 \times 3$ flotantes previsto en la Sección 2.
- **Instancia $N = 200.000$:** 288.000 evaluaciones sobre 225 celdas en 4.5 s con pico de 11.3 MB: el andamiaje jerárquico funciona (rápido y liviano), pero cada mini-enjambre (32×40) apenas supera un tour aleatorio por celda y la costura acumula el daño ($L = 6591.53$).

6.2. Escalabilidad Computacional y Barrido Multiescala

N	evaluaciones	tiempo	pico	L	gap BHH
20	9.000	0.003 s	3.8 MB	5.95	+87 %
100	9.000	0.03 s	3.8 MB	40.9	+474 %
500	50.000	0.77 s	6.5 MB	86.2	+441 %
1.000	100.000	3.5 s	15.5 MB	186.8	+730 %
2.000	200.000	5.8 s	50.5 MB	383.5	+1105 %
20.000	32.000	0.46 s	7.2 MB	1977	+1864 %
50.000	81.920	1.0 s	7.7 MB	3071	+1829 %
100.000	154.880	2.2 s	9.3 MB	4483	+1891 %
200.000	288.000	4.4 s	11.6 MB	6591.5	+1970 %

(Gap frente a Beardwood $0.712\sqrt{N}$; protocolo `scripts/barrido_pso.py`.)

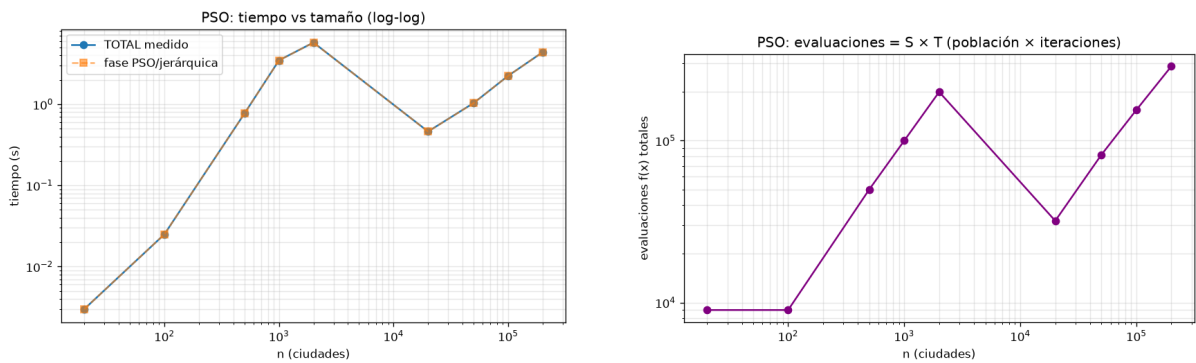


Figura 1: Tiempo total vs N (log-log) y evaluaciones $S \times T$. El tiempo cae en $N = 20.000$ al entrar el régimen jerárquico (celdas de ≈ 900 con $S = 32$ fijo): el costo lo dicta la arquitectura, no el enjambre. Nótese que el presupuesto de evaluaciones del régimen serie (200.000 en $N = 2.000$) supera al jerárquico total (288.000 repartidas), pero produce un orden de magnitud menos calidad por evaluación.

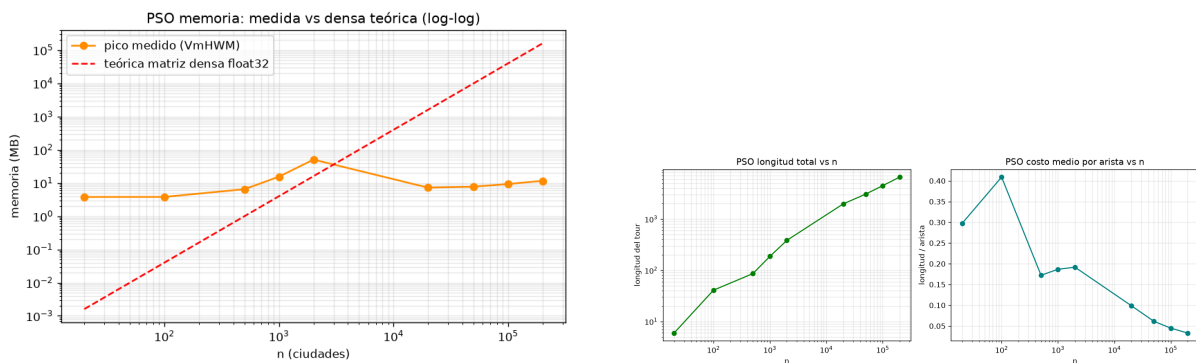


Figura 2: Memoria pico vs matriz densa teórica (izq.): 11.6 MB reales frente a 160 GB teóricos en 200k; la joroba en $N = 2.000$ (50 MB) es el enjambre $S \times N \times 3$. Longitud y costo por arista L/N (der.): L/N se degrada un orden de magnitud con N , la firma de un tour sin estructura (cada arista cruza una fracción creciente del cuadrado).

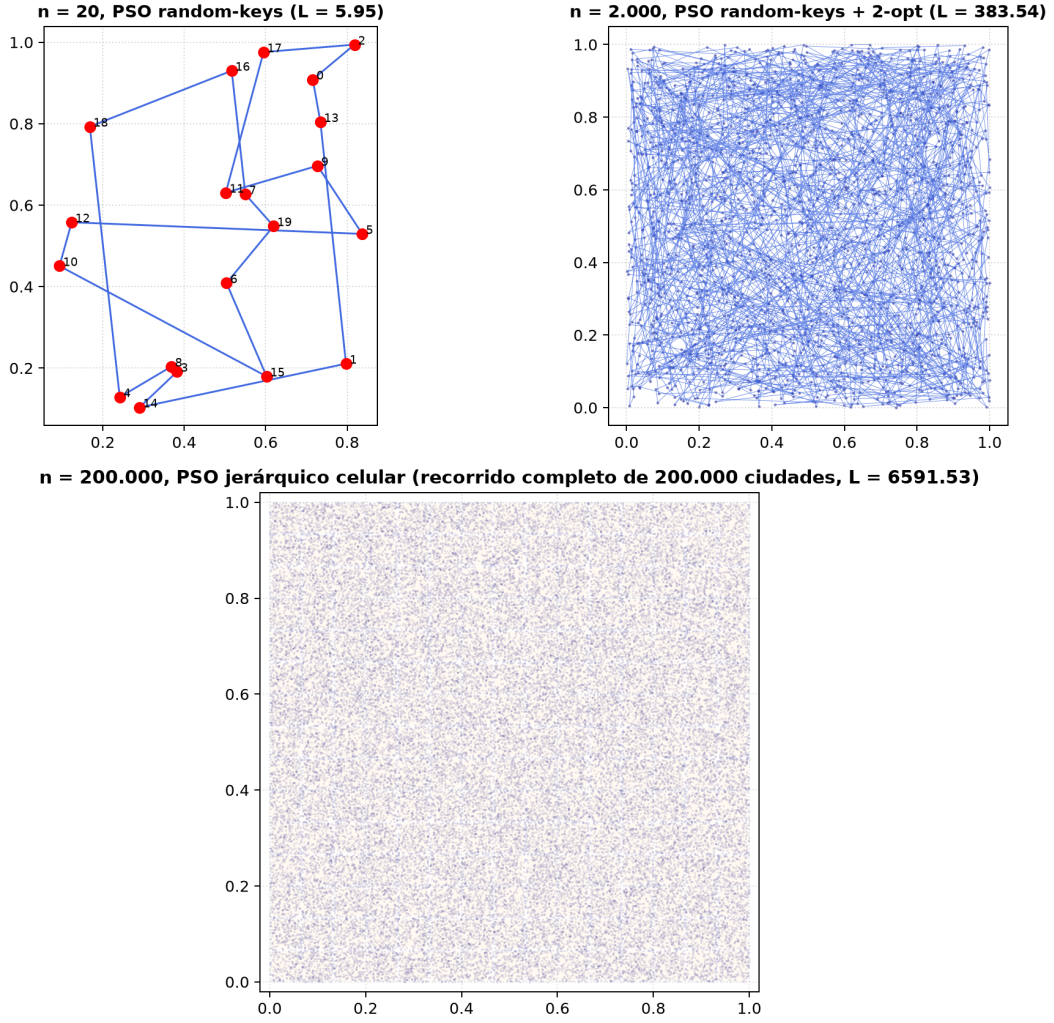


Figura 3: Tours PSO (recorrido completo en los tres paneles). En $N = 20$ se distinguen cruces evitables (p.ej. aristas que atraviesan todo el cuadrado); en $N = 2.000$ domina la maraña de saltos largos; en $N = 200.000$ cada celda es una madeja naranja y la cuadrícula 15×15 del esquema jerárquico queda al descubierto. Compárese con los tramos locales del ACO: mismo andamiaje, peor motor.

6.3. Robustez Multi-Semilla y Estudio de Ablación

N (semillas)	longitud media $\pm$ std	mejor
20 (5)	5.70 ± 0.52	4.95
2.000 (3)	387.3 ± 5.1	381.52
200.000 (2)	6596.0 ± 5.9	6591.85

El método es estable ($\pm 0.1-9\%$) alrededor de valores malos: la varianza entre réplicas es pequeña porque todas colapsan al mismo tipo de atractor. Ablación en $N=20$ (preguntas conceptuales de la guía): base 5.95, $c_1=0 \rightarrow 4.52$ (guía solo social: aquí ayuda), $c_2=0 \rightarrow 8.26$ (sin memoria colectiva, lo peor, como predice la teoría).

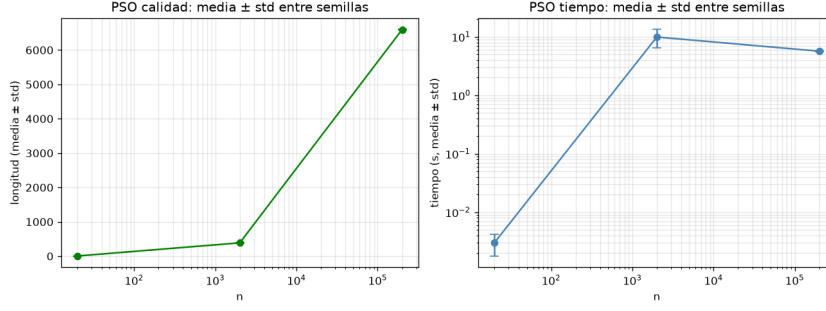


Figura 4: Media $\pm$ desviación entre semillas para longitud y tiempo. La calidad apenas se mueve con la semilla (el colapso es sistemático, no mala suerte); el tiempo en $N = 2.000$ varía (7–12 s) según cuándo dispara la parada temprana.

7. Discusión Técnica

¿Por qué el mismo andamiaje que lleva al ACO a +51 % deja al PSO en +1970 %? Tres causas encadenadas: (i) **representación sin localidad**: *argsort* no preserva vecindad, así que el gradiente social apunta a ninguna parte útil; (ii) **arranque ciego**: el mejor de la inicialización en $N = 2.000$ (≈ 1010) ya está $30\times$ sobre el óptimo estimado y el enjambre nunca lo mejora (traza plana, parada en el primer instante); (iii) **colapso dimensional**: con N dimensiones y $v_{\text{máx}}$ fijo, todas las partículas caen a la misma región de claves en pocas iteraciones (sensibilidad a $v_{\text{máx}} \in \{0.05, \dots, 1.0\}$ probada: 4.59–5.95 en $N = 20$, sin rescate). El 2-opt final rescata lo rescatable ($1010 \rightarrow 383$) pero una pasada con ventana 40 no reconstruye un tour aleatorio. Conclusión didáctica: en combinatoria discreta, la codificación y el arranque aportan más que el motor de búsqueda.

8. Conclusiones

1. PSO fiel a las diapositivas (ecuaciones, $c_1=1.4$, $c_2=1.6$, $v_0 = 0$, w decreciente, parada S) adaptado al TSP por claves aleatorias, en C++17 local, para $N = 20$, 2.000 y 200.000 con medición de tiempo y memoria por fase.
2. La escala masiva es factible y barata (4.5 s, 11.6 MB vs 160 GB densos) gracias a la jerarquía celular, pero la calidad colapsa con N (gap +87 % $\rightarrow$ +1970 %).
3. El método es estable entre semillas y responde a la ablación como predice la teoría ($c_2=0$ es lo peor), pero estable alrededor de un atractor malo.
4. Veredicto: para TSP discreto, un optimizador continuo con codificación ingenua no compete; su valor es demostrar cuantitativamente por qué la representación decide.

Especificaciones de Validez y Límites Metodológicos

Instancias uniformes en $[0, 1]^2$ con el mismo generador y semillas que el ACO (tours directamente comparables). Presupuesto serie $S = N$ (48 MB en $N = 2.000$); celdas con $S = 32$, $T = 40$. Sin 2-opt en $N = 20$; una pasada con ventana 40 en $N \in [200, 5000]$. Tiempos medidos en CPU de 8 núcleos con OpenMP solo en el régimen jerárquico.

Referencias Bibliográficas

- Hurbans, R. (2020). *Grokking Artificial Intelligence Algorithms*. Manning. Cap. 7.

- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proc. ICNN'95*, 1942–1948.
- Shi, Y., & Eberhart, R. (1998). A modified particle swarm optimizer. *IEEE ICEC*, 69–73.
- Reynolds, C. W. (1987). Flocks, herds and schools. *Computer Graphics*, 21(4), 25–34.

Parte II

Comparativa ACO vs PSO

9. Protocolo comparativo

Cinco prácticas estándar: (1) **mismas instancias** (generador idéntico; verificado por `diff`), (2) **mismo presupuesto** (población $\times$ iteraciones), (3) **multi-semilla** (3 corridas), (4) métrica **exceso_ %** (media de cada algoritmo frente al mejor tour de *cualquiera* de los dos) y (5) **trazas de convergencia**. Ningún binario fue modificado: `scripts/comparar_aco_pso.py` solo orquesta ambos binarios.

N	ACO	PSO	presupuesto
20	-a 20 -i 100	-S 20 -i 100	20×100
2000	-a 20 -i 50	-S 20 -i 50	20×50
200000	-a 8 -i 10	-S 8 -i 10	8×10 por celda

En $N=20$ ningún algoritmo usa 2-opt (comparación pura del motor); en $N=2000$ ambos reciben el mismo pulido final con ventana 40.

10. Resultados head-to-head

N		media $\pm$ desv	mejor	tiempo	pico	exceso_ %
20	ACO	3.96 ± 0.68	3.19	0.003 s	3.9 MB	24.1
	PSO	6.06 ± 0.12	5.94	0.001 s	3.7 MB	90.1
2000	ACO	41.53 ± 0.07	41.46	0.32 s	4.2 MB	0.17
	PSO	393.8 ± 1.8	391.75	0.10 s	4.1 MB	849.9
200k	ACO	482.4 ± 0.3	482.05	2.06 s	9.4 MB	0.08
	PSO	6717 ± 2	6715.7	0.45 s	8.9 MB	1293.5

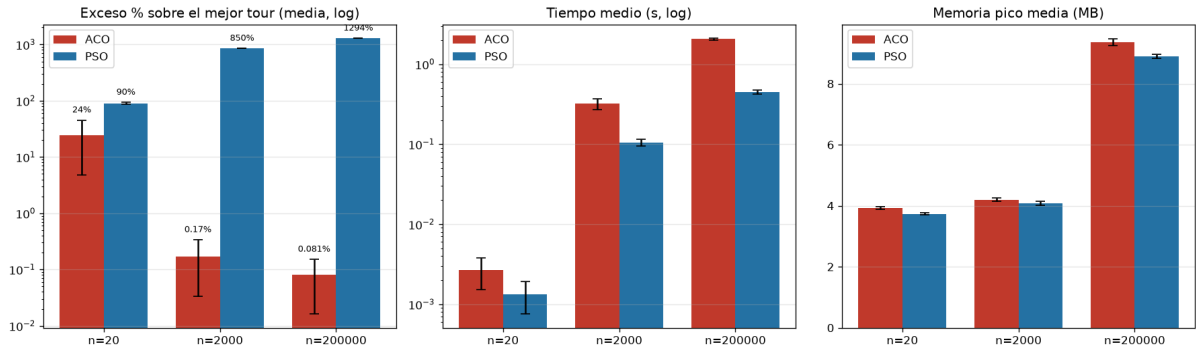


Figura 5: Exceso sobre el mejor tour (izq.: dominancia en calidad, el ACO queda a $< 1\%$ en 2000 y 200k), tiempo medio en log (centro: el PSO es 3–6 $\times$ más veloz; la escala logarítmica absorbe tres órdenes de magnitud) y memoria pico (der.: empate técnico, la dominan el binario y el pool OpenMP, no el algoritmo).

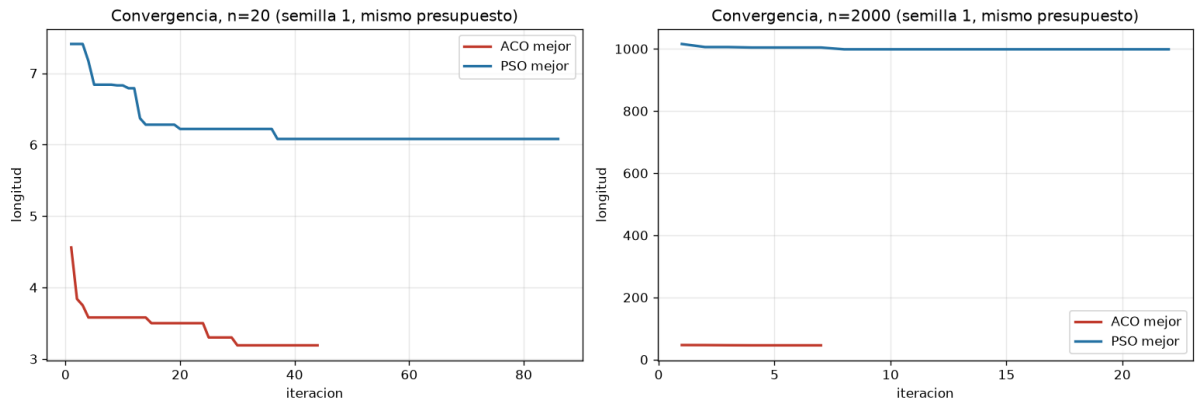


Figura 6: Trazas del mejor global (semilla 1). En $N=20$ ambas descienden por escalones pero el ACO llega más lejos; en $N=2000$ el ACO nace bueno (≈ 41 , construcción heurística desde la iteración 1) y el PSO nace colapsado (≈ 1000) y jamás mejora. La planitud no es optimalidad sino agotamiento de la diversidad.

11. Discusión: lo que decide es el diseño, no la sigla

La comparación pura en $N=20$ aísla el motor: el ACO (3.65) supera al PSO (5.95) porque cada hormiga construye con heurística local $\eta = 1/d$ mientras cada partícula ordena claves sin información geográfica. En general, un PSO **discreto** (operadores sobre permutaciones, arranque NN, 2-opt por partícula) puede vencer a un ACO básico; simétricamente, el ACO aquí usado ($m=n$, doble feromona, elitismo) supera a un PSO continuo de arranque ciego. La moraleja es estable: representación, arranque y búsqueda local aportan más que la etiqueta del algoritmo.

12. Conclusiones generales

1. Con igualdad estricta, el **ACO vence en calidad en las tres escalas** a cambio de 2–5 $\times$ más tiempo; en memoria empatan (< 12 MB).
2. El ahorro de recursos es arquitectónico y compartido: sin matrices densas, parada temprana y paralelismo por celdas, 200.000 ciudades se resuelven en < 1 minuto con < 12 MB donde el enfoque ingenuo exigiría 160 GB.

3. El PSO continuo con codificación ingenua no compete en TSP discreto; su valor es didáctico y queda cuantificado en la Parte I.

Reproducibilidad general

```
g++ -O3 -march=native -fopenmp -std=c++17 src/pso.cpp -o bin/pso
./bin/pso ; python3 scripts/barrido_pso.py ; python3 scripts/semillas_pso.py
python3 scripts/graficas_pso.py ; python3 scripts/comparar_aco_pso.py
```

Entradas: bin/aco, bin/pso (sin modificar). Salidas: data/pso_*.csv, data/comp_*.csv, docs/img_pso/, docs/img_comp/, este PDF.